

EpiTunes

Documentazione tecnica completa

Architettura, struttura dei file, flusso dati, classi JavaScript,
localStorage, layout responsive e logica di ogni pagina

Generato il 25/06/2026 — basato sullo stato attuale del codice nel repository

EpiTunes — Documentazione Tecnica

1. Panoramica del progetto

EpiTunes è un'applicazione web multi-pagina (non una SPA) che replica l'interfaccia e le funzionalità essenziali di un music player in stile Spotify, usando esclusivamente **HTML, CSS e JavaScript vanilla** (nessun framework, nessun bundler, nessun passo di build). I dati sui brani, album e artisti vengono recuperati in tempo reale dalla **iTunes Search/Lookup API** pubblica di Apple, che fornisce anche un URL di anteprima audio di 30 secondi per ogni brano (`previewUrl`), riprodotto tramite l'elemento HTML `<audio>` .

Non esiste un backend proprietario: tutta la persistenza (cronologia, preferiti, playlist, volume, stato shuffle/repeat, ultima ricerca) avviene nel `localStorage` del browser. Questo significa che i dati sono legati al singolo browser/dispositivo e non sono condivisi tra utenti o sincronizzati altrove.

1.1 Stack tecnologico

Componente	Dettaglio
Markup	5 pagine HTML statiche indipendenti (index, album, artist, playlist, search), ciascuna con la stessa shell (sidebar + topbar + footer player)
Stile	Bootstrap 5.3.3 (via CDN jsdelivr) per griglia, utility, offcanvas, collapse + <code>assets/css/app.css</code> per il tema scuro custom e il layout a 3 zone
Icone	Ionicons 7.1.0 (via CDN unpkg), caricate come modulo ES con fallback <code>nomodule</code>
Logica	JavaScript ES6+ vanilla: classi, arrow function, <code>fetch / async-await</code> , template literals, <code><template></code> + <code>cloneNode</code>
Dati esterni	iTunes API (<code>https://itunes.apple.com/search</code> e <code>/lookup</code>), nessuna API key richiesta
Persistenza	<code>window.localStorage</code> , sempre stringhe (JSON.stringify/parse per gli array di oggetti)
Audio	Un singolo elemento <code><audio id="audio-element"></code> per pagina, pilotato dalla classe <code>Player</code>

2. Struttura dei file

```
BUILD-WEEK-2/
├─ index.html      → Home (righe: cronologia, preferiti, suggerimenti pop/rock/hits)
├─ search.html     → Ricerca live (brani, album, artisti)
├─ album.html      → Dettaglio album (hero + tracklist)
├─ artist.html     → Dettaglio artista (hero + top tracks)
├─ playlist.html   → Dettaglio playlist personale o "preferiti"
├─ README.md
├─ assets/
│   └─ css/app.css → Unico file di stile custom (821 righe)
```

```

├── img/epicode-logo.png
└── js/
    ├── common.js → Codice condiviso da TUTTE le pagine (1125 righe) – cuore del sistema
    ├── home.js   → Logica esclusiva di index.html
    ├── search.js → Logica esclusiva di search.html
    ├── album.js  → Logica esclusiva di album.html
    ├── artist.js → Logica esclusiva di artist.html
    └── playlist.js → Logica esclusiva di playlist.html

```

Ogni pagina HTML carica gli script nello stesso ordine fisso:

```

<script src="assets/js/common.js"></script>
<script src="assets/js/<nome-pagina>.js"></script>

```

Questo ordine è **strutturale**: `common.js` definisce costanti globali (`const` a livello di file, quindi visibili come variabili globali dato che gli script non sono moduli), classi (`Track` , `Album` , `Artist` , `Player`) e funzioni (`fetchJSON` , `initPage` , `getFavourites` , ecc.) che gli script di pagina usano immediatamente, senza alcun import: essendo tutti script classici (non `type="module"`), condividono lo stesso scope globale del documento.

3. Layout CSS a 3 zone e responsive

Il contenitore radice `.app` (in ogni pagina) è una **CSS Grid** che definisce tre aree: `sidebar` , `main` , `player` . La definizione cambia a seconda della larghezza dello schermo, secondo un approccio **mobile-first** (gli stili di base si applicano allo schermo più piccolo, poi le media query `min-width` aggiungono/sovrascrivono regole per schermi più grandi).

Breakpoint	Comportamento
< 576px (default, nessuna media query)	<code>.sidebar</code> ha <code>display: none</code> ; <code>.app</code> usa <code>grid-template-columns: 1fr</code> con righe <code>main + player</code> . Il menu di navigazione è sostituito dall' offcanvas Bootstrap (pannello laterale apribile dall'hamburger). La barra del volume nel player (<code>.player-right</code>) è nascosta (<code>display: none</code>): sotto questa soglia il volume audio è gestito dai tasti fisici del dispositivo (vedi sezione 7.6).
≥ 576px	<code>.app</code> passa a <code>grid-template-columns: 80px 1fr</code> con aree <code>"sidebar main" / "player player"</code> . La sidebar appare ma solo a icone (niente testo: <code>.brand-text</code> , <code>.sidebar-section-title</code> , <code>.sidebar-list</code> e le etichette testuali dei link restano <code>display: none</code>). <code>.player-right</code> torna visibile (<code>display: flex</code>): da qui in su la barra del volume è disponibile.
≥ 768px	<code>.app</code> passa a <code>grid-template-columns: 240px 1fr</code> : la sidebar diventa "piena", mostrando testo (<code>.brand-text</code> , le label dei link), il titolo "Filtri" e le liste Preferiti/Playlist.

Le classi utility Bootstrap `d-none d-sm-block` e `d-sm-none d-flex` sulla topbar gestiscono separatamente la visibilità del pulsante utente (desktop, da 576px) contro l'hamburger del menu

mobile (solo sotto 576px) — un sistema di breakpoint paritetico a quello della sidebar in `app.css`, ma espresso con le classi di Bootstrap direttamente nell'HTML invece che con media query custom.

4. Costanti e helper globali (common.js, sezioni 1-2)

4.1 Costanti (righe 61-70)

Costante	Valore	Uso
<code>API_BASE</code>	<code>https://itunes.apple.com</code>	Prefisso per tutte le chiamate <code>fetchJSON</code>
<code>STORAGE_KEY_HISTORY</code>	<code>epitunes_history</code>	Array JSON di Track ascoltati di recente (max <code>MAX_HISTORY</code> = 12)
<code>STORAGE_KEY_FAVOURITES</code>	<code>epitunes_favourites</code>	Array JSON di Track preferiti
<code>STORAGE_KEY_PLAYLIST</code>	<code>epitunes_playlist</code>	Vecchia chiave per UNA sola playlist (oggi usata solo come fonte di migrazione, vedi 6.3)
<code>STORAGE_KEY_PLAYLISTS</code>	<code>epitunes_playlists</code>	Array JSON di playlist multiple: [{id, name, tracks}]
<code>STORAGE_KEY_LAST_SEARCH</code>	<code>epitunes_last_search</code>	Ultima query digitata in ricerca, ripristinata al ricaricamento di search.html
<code>STORAGE_KEY_VOLUME</code>	<code>epitunes_volume</code>	Volume (0-1) persistito tra sessioni desktop/tablet
<code>STORAGE_KEY_SHUFFLE</code>	<code>epitunes_shuffle</code>	Stringa "true" / "false" per lo stato shuffle
<code>STORAGE_KEY_REPEAT</code>	<code>epitunes_repeat</code>	Stringa "true" / "false" per lo stato repeat (mappato su <code>audio.loop</code>)

4.2 Helper di rete e formattazione (righe 80-127)

- `fetchJSON(url)` — wrapper attorno a `fetch`: lancia un errore se `response.ok` è falso, altrimenti ritorna `response.json()`. In caso di qualunque eccezione (rete assente, HTTP non-2xx, JSON malformato), il `catch` stampa l'errore in console e la funzione ritorna comunque `{ results: [], resultCount: 0 }`, così tutto il codice chiamante può sempre fare `.results.map(...)` senza controlli aggiuntivi su `undefined`.
- `bigArt(url)` — l'API iTunes restituisce sempre artwork a 100×100px nel campo `artworkUrl100`; la funzione sostituisce la sottostringa "100x100" con "600x600" nell'URL per ottenere una cover ad alta risoluzione (usata nelle hero di album/artista/playlist e nelle card di ricerca).
- `formatTime(ms)` — converte millisecondi in stringa `m:ss` (es. 65000 → "1:05"); ritorna "0:00" se l'input è falsy.
- `debounce(fn, ms)` — pattern classico: ritorna una funzione che ad ogni chiamata cancella un timer precedente (`clearTimeout`) e ne pianifica uno nuovo; `fn` viene eseguita solo se passano `ms` millisecondi senza nuove chiamate. Usata con 400ms su tutte le searchbar delle pagine.

5. Sistema filtri sidebar (myFunction / renderResultados, righe 136-272)

I tre badge "Artisti / Album / Generi" nella sidebar (visibili solo da 768px, classe `d-none d-md-flex`) non interrogano l'API: ricavano i dati esclusivamente dai **preferiti già salvati** (`getFavourites()`), raggruppandoli:

- **Artisti**: una `Map` indicizzata per `track.artistId`, un solo elemento per artista, con cover presa dal primo brano preferito di quell'artista.
- **Album**: stessa logica ma indicizzata per `track.albumId`.
- **Generi**: indicizzata per `track.genre` (fallback "Sconosciuto" se assente).

La variabile globale `filtroActivo` tiene traccia di quale filtro è attivo: un secondo click sullo stesso badge lo disattiva (`resetFiltros()`: svuota `#sidebar-filter-results` e rimette tutti i badge a `bg-secondary`); cliccare un badge diverso disattiva automaticamente quello precedente, garantendo che ne sia attivo **al massimo uno alla volta** (badge attivo colorato con la classe Bootstrap `bg-success`).

`renderResultados(lista, tipo)` clona il `<template>` appropriato (`#tmpl-album-item`, `#tmpl-artist-item` o `#tmpl-filter-result`, tutti definiti nell'HTML di ogni pagina) per ciascun elemento della lista raggruppata, e aggiunge un listener di click che naviga a `album.html?id=...` o `artist.html?id=...` (i generi non sono cliccabili, mostrano solo l'etichetta).

6. Classi modello: Track, Album, Artist (righe 284-317)

Queste tre classi sono semplici **adattatori**: prendono l'oggetto grezzo (`raw`) restituito dall'API iTunes e lo rimappano su nomi di campo più leggibili e stabili, usati da tutto il resto del codice invece dei nomi originali dell'API.

Classe	Campo interno	Campo iTunes originale
Track	id	trackId
	title	trackName
	artist	artistName
	album	collectionName
	albumId	collectionId
	artistId	artistId
	cover	artworkUrl100
	previewUrl	previewUrl (anteprima audio 30s)
	durationMs / genre	trackTimeMillis / primaryGenreName
Album	id	collectionId

	title / artist / artistId	collectionName / artistName / artistId
	cover	artworkUrl100
	releaseDate	releaseDate
	trackCount	trackCount
Artist	id	artistId
	name	artistName
	genre	primaryGenreName

Ogni `Track` degli oggetti salvati in `localStorage` (cronologia, preferiti, playlist) è un oggetto JSON semplice creato da `new Track(raw)` e poi serializzato — la classe stessa non viene mai serializzata, solo i suoi campi.

7. La classe Player — cuore del lettore audio (righe 340-753)

Una singola istanza di `Player` viene creata **una volta per pagina** da `initPage()` e assegnata a `window.player`, così è raggiungibile da qualunque script di pagina (es. `window.player.play(track)`). Lo stato del player (brano corrente, playing/paused, shuffle pool) **non sopravvive alla navigazione tra pagine**: ogni volta che si cambia pagina HTML, viene creata una nuova istanza e l'audio ricomincia da fermo — solo i flag persistenti in `localStorage` (volume, shuffle, repeat) vengono ripristinati.

7.1 Costruttore (righe 341-384)

Recupera l'elemento `<audio id="audio-element">` presente in ogni pagina (`this.audio`). Se non esiste, il costruttore si interrompe subito (`return`) lasciando lo stato minimo: questo rende la classe sicura da istanziare anche su pagine ipotetiche senza tag audio. Inizializza:

- `currentTrack = null`, `isPlaying = false`
- `currentTracklist = []` — l'elenco di brani della "riga"/album/artista/playlist da cui si è partiti, usato da `next()` / `prev()`
- `isShuffle` e `isRepeat` letti subito da `localStorage` (persistenza cross-pagina)
- `shufflePool = []` — pool di id non ancora suonati in modalità shuffle
- `volumeBeforeMute = 0.5` — fallback iniziale prima che `setVolume` venga chiamato
- `this.audio.loop = this.isRepeat` — il repeat è delegato direttamente alla proprietà nativa `loop` dell'elemento `<audio>`, non reimplementato manualmente

Registra due listener permanenti sull'audio:

- `timeupdate` — ad ogni avanzamento della riproduzione, aggiorna il testo di `#time-current` e la larghezza (%) di `#progress-fill` in base a `currentTime / duration`.
- `ended` — se `currentTracklist.length > 1` (siamo dentro un album/playlist/riga con più brani) chiama `this.next()` per passare automaticamente al brano successivo; altrimenti riporta semplicemente `isPlaying = false`, ripristina l'icona ▶ e chiama `updateNowPlayingUI()`.

7.2 mount() — costruzione del footer player (righe 386-565)

Questo metodo **costruisce da zero, via `document.createElement`** (nessun `innerHTML`), tutta l'interfaccia del player nel `<footer class="player">`, che nell'HTML di partenza è vuoto. Viene chiamato una sola volta da `initPage()`. Struttura creata, da sinistra a destra (corrispondente alle 3 colonne della grid `.player` definita in CSS, `1fr 2fr 1fr`):

Zona	Elementi creati	id / classi
player-track (sinistra)	cover (img), titolo (<code><a></code> , non <code><p></code> : il click porta su <code>album.html</code>), artista (<code><a></code> verso <code>artist.html</code>)	<code>#player-cover-img</code> , <code>#player-title</code> , <code>#player-artist</code>
player-center	controlli (shuffle, prev, toggle play/pause, next, repeat) + barra di	<code>#btn-shuffle</code> , <code>#btn-prev</code> , <code>#btn-toggle</code> , <code>#btn-next</code> , <code>#btn-repeat</code> , <code>#progress-bar</code> / <code>#progress-</code>

(centro)	progresso con tempo corrente/totale	<code>fill</code> , <code>#time-current / #time-total</code>
player-right (destra)	icona muto + barra volume (nascosta sotto 576px, vedi 7.6)	<code>#btn-mute</code> , <code>#volume-bar / #volume-fill</code>

Dopo aver inserito i tre blocchi nel footer con `footer.replaceChildren(track, center, right)` , il metodo:

1. Applica lo stato iniziale del volume (vedi 7.6).
2. Aggancia **tutti** i listener di interazione: `click` su `btnToggle` → `togglePlay()` ; `mousedown` su `progressBar / volumeBar` con un sistema di drag condiviso (vedi 7.3); `click` su `volumeIcon` → `toggleMute()` ; `click` su `prev/next/shuffle/repeat` → i metodi omonimi del Player.

7.3 Drag delle barre (progress e volume, righe 520-555)

Una funzione condivisa `percentFromEvent(bar, e)` calcola la posizione orizzontale del cursore (`e.clientX`) relativa al rettangolo della barra (`getBoundingClientRect()`), clampata tra 0 e 1. Due flag booleani indipendenti, `isDraggingProgress` e `isDraggingVolume`, vengono attivati su `mousedown` sulla rispettiva barra. Due listener **globali sul document** (non sulla barra stessa) gestiscono `mousemove` (aggiorna la barra il cui flag è attivo) e `mouseup` (azzerà entrambi i flag): questo è il motivo per cui il drag **continua a funzionare anche se il mouse esce dai confini della barra** durante il trascinamento — pattern standard per slider custom in vanilla JS.

Nota tecnica: i listener sono basati su eventi `mouse*` puri, quindi su dispositivi touch (mobile/tablet) il drag della progress bar e della volume bar non risponde al tocco — non sono gestiti eventi `touchstart / touchmove / pointer*`. Su mobile l'unico modo per cambiare posizione nel brano resta indiretto.

7.4 play(track, tracklist) (righe 567-598)

Punto di ingresso principale per avviare la riproduzione, chiamato da ogni pagina (card, riga di tracklist, bottone play grande, voce preferiti in sidebar, ecc.). Se `track` è assente o non ha `previewUrl`, esce immediatamente senza fare nulla (i brani senza anteprima — che l'API iTunes a volte restituisce — sono quindi silenziosamente non riproducibili). Se viene passato un `tracklist` non vuoto, **sovrascrive** `this.currentTracklist`: questo è il meccanismo con cui ogni pagina comunica al player "in che contesto" si trova il brano, abilitando `next()` / `prev()` a scorrere quella stessa lista (es. tutte le tracce di un album, o tutti i brani di una riga della Home).

Imposta `audio.src = track.previewUrl`, chiama `audio.play()`, marca `isPlaying = true`, e aggiorna tutta la UI del footer (cover, titolo con link ad album, artista con link ad artista, tempo totale, icona toggle a `II`). Se la funzione globale `addToHistory` esiste, registra il brano in cronologia. Chiude chiamando `updateNowPlayingUI()`.

7.5 togglePlay(), setCardPlayIcon(), updateNowPlayingUI() (righe 600-651)

`togglePlay()` non fa nulla se `currentTrack` è `null` (nessun brano caricato); altrimenti alterna `audio.pause()` / `audio.play()` e l'icona del bottone centrale tra  e .

`setCardPlayIcon(btnPlay, isPlaying)` gestisce la doppia natura del bottone play sulle card: nella Home (`home.js`) il bottone contiene un `<ion-icon>` (cambia l'attributo `name` tra `play-outline` / `pause-outline`), mentre in Ricerca (`search.js`) è testo semplice  /  — la funzione controlla la presenza dell'icona e si comporta di conseguenza, evitando di sovrascrivere sempre con `textContent`.

`updateNowPlayingUI()` è chiamata dopo ogni cambio di stato di riproduzione: prima rimuove la classe `is-playing` da **tutte** le righe tracklist (`.track-row.is-playing`) e da tutte le card (`.card-play.is-playing`, ripristinando anche l'icona a "play"); poi, se esiste un `currentTrack`, ri-applica `is-playing` a tutti gli elementi con `data-id` uguale all'id del brano corrente — usando `querySelectorAll` invece di un singolo elemento perché lo stesso brano può comparire contemporaneamente in più righe della Home (es. sia in "Riprodotti di recente" che in "Suggerimenti pop").

7.6 setVolume(), toggleMute() e gestione del volume responsive (righe 500-516, 652-678)

Questa è la parte più recentemente modificata del file. `setVolume(v)` imposta `audio.volume = v` (clampato 0-1 dai chiamanti), aggiorna la larghezza visiva di `#volume-fill`, sceglie l'icona del bottone muto ( se 0,  se < 0.5 ,  altrimenti) e **persiste** il valore in `localStorage` tramite `STORAGE_KEY_VOLUME` — questo è ciò che fa "ricordare" il volume tra una sessione e l'altra su desktop. Se $v > 0$ aggiorna anche `volumeBeforeMute`, in modo che il volume di riferimento per il "ripristino dal muto" sia sempre l'ultimo valore non-zero impostato manualmente, anche trascinando la barra (non solo passando dal bottone muto).

`toggleMute()` è un semplice toggle: se il volume attuale è > 0 , lo salva in `volumeBeforeMute` e chiama `setVolume(0)`; altrimenti richiama `setVolume(volumeBeforeMute || 0.5)` (il fallback 0.5 copre il caso limite in cui `volumeBeforeMute` sia ancora 0).

Comportamento responsive del volume (righe 500-516): all'interno di `mount()` viene creato un `MediaQueryList` tramite `window.matchMedia("(max-width: 575.98px)")` — la stessa soglia di 576px usata in `app.css` per nascondere `.player-right`. La funzione `applyVolumeForViewport()`:

- se il viewport corrisponde a "mobile" (sotto 576px), forza `this.audio.volume = 1` **senza** passare per `setVolume()` (quindi senza toccare `localStorage` né la UI della barra, che in questa fascia è comunque invisibile);
- altrimenti, legge il volume salvato precedentemente da `localStorage`, lo clampa tra 0 e 1 (con fallback 0.5 se assente o non numerico), e lo applica con `this.setVolume(initialVolume)` aggiornando anche la UI.

Viene invocata una volta subito dopo il montaggio del footer, e poi di nuovo automaticamente ogni volta che il `MediaQueryList` cambia stato (evento `change`, scatenato sia da resize della finestra che da rotazione del dispositivo), grazie a `mobileVolumeQuery.addEventListener("change", applyVolumeForViewport)`. **Razionale:** sotto i 576px la barra del volume è nascosta dal CSS (`.player-right { display: none }`), con eccezione a partire da 576px dentro la media query, quindi non ha

senso lasciare l'elemento `<audio>` a un volume parziale (es. 0.3 salvato da una sessione desktop precedente): forzandolo al massimo, il volume percepito dall'utente mobile è determinato esclusivamente dal volume di sistema del dispositivo, pilotabile con i tasti fisici hardware.

7.7 seek(), toggleShuffle(), toggleRepeat() (righe 680-703)

`seek(percent)` non fa nulla se l'audio non ha ancora una `duration` nota (cioè nessun metadato caricato); altrimenti imposta `audio.currentTime = percent * audio.duration`.

`toggleShuffle()` inverte `isShuffle`, lo persiste in `localStorage`, aggiorna il colore del bottone (verde #1db954 se attivo) e, se ora è attivo, richiama `initShufflePool()` per popolare immediatamente il pool.

`toggleRepeat()` inverte `isRepeat`, lo persiste, e — punto importante — aggiorna direttamente `this.audio.loop`: il repeat-singolo-brano è quindi delegato al comportamento nativo del tag `<audio>`, non gestito manualmente nell'handler `ended`.

7.8 Shuffle pool, next() e prev() (righe 705-752)

`initShufflePool()` popola `shufflePool` con gli id di tutti i brani di `currentTracklist` **escluso** quello corrente. `next()`:

- se la tracklist corrente ha 1 o 0 elementi, non c'è un "prossimo" reale: si limita a riportare il brano corrente all'inizio (`seek(0)`);
- in modalità shuffle: se il pool è vuoto lo ricostruisce (gestendo anche il caso limite di tracklist con un solo elemento, in cui torna a `seek(0)`); altrimenti estrae un indice casuale dal pool, lo rimuove (`splice` , garantendo di non ripetere lo stesso brano finché il pool non si esaurisce) e riproduce quel brano;
- in modalità lineare: trova l'indice del brano corrente in `currentTracklist` e passa al successivo con wrap-around (`% length` , quindi dopo l'ultimo brano si torna al primo).

`prev()` ha una particolarità "stile Spotify": se sono già passati più di 3 secondi di riproduzione del brano corrente (`audio.currentTime > 3`), il bottone "precedente" riporta semplicemente all'inizio del brano corrente invece di saltare al brano prima nella lista; solo se si preme entro i primi 3 secondi (o se la tracklist ha 1 elemento) si passa effettivamente al brano precedente, con lo stesso wrap-around circolare di `next()` . La modalità `prev` **non** rispetta lo shuffle: scorre sempre linearmente all'indietro nella tracklist, indipendentemente da `isShuffle` .

8. localStorage: cronologia, preferiti, playlist (righe 755-981)

8.1 Cronologia (righe 765-782)

`getHistory()` legge e fa il parse JSON della chiave `epitunes_history` (array vuoto se assente). `addToHistory(track)` è chiamata automaticamente da `Player.play()` ad ogni riproduzione: rimuove eventuali duplicati dello stesso `track.id` già presenti, inserisce il nuovo brano in testa (`unshift` , quindi i più recenti compaiono primi) e tronca l'array a `MAX_HISTORY = 12` elementi.

8.2 Preferiti (righe 784-806)

`getFavourites()` / `isFavourite(trackId)` sono helper di sola lettura. `toggleFavourite(track)` verifica se il brano è già presente per `id` : se sì lo rimuove (`filter`), se no lo aggiunge in testa (`unshift`); salva sempre il risultato e — punto rilevante — chiama subito `renderSidebarFavourites()` , così la lista "Preferiti" nella sidebar (sia desktop che offcanvas mobile) si aggiorna **immediatamente** in tutte le pagine senza bisogno di un ricaricamento, ogni volta che un cuoricino viene cliccato ovunque nell'app.

8.3 Playlist singola legacy e migrazione (righe 808-829, 877-889)

`getPlaylist()` / `isInPlaylist()` / `togglePlaylist()` operano sulla vecchia chiave `STORAGE_KEY_PLAYLIST` (singolare), risalente a una versione precedente del progetto in cui esisteva una sola playlist personale. `migrateOldPlaylist()` , eseguita una volta ad ogni `initPage()` , controlla se questa vecchia chiave contiene ancora dati: se sì e non è già stata fatta una migrazione precedente (verificato cercando una playlist con `id === "migrated"` tra quelle nuove), crea una playlist multipla con `id: "migrated"` e `name: "La tua playlist"` contenente quei brani, la salva nella nuova chiave `STORAGE_KEY_PLAYLISTS` , e **rimuove** la vecchia chiave (`localStorage.removeItem`) per non rieseguire la migrazione in futuro.

8.4 Playlist multiple (righe 831-981)

La struttura dati corrente è un array di oggetti `{id, name, tracks}` sotto `STORAGE_KEY_PLAYLISTS` . Funzioni principali:

- `getPlaylists()` / `getPlaylistById(id)` — lettura
- `createPlaylist(name)` — genera un id univoco basato sul timestamp corrente (`pl_${Date.now()}`), crea la playlist vuota, salva e richiama `renderSidebarPlaylists()` per aggiornare subito la sidebar
- `deletePlaylist(id)` — filtra via la playlist con quell'id, salva, aggiorna la sidebar
- `toggleTrackInPlaylist(playlistId, track)` — trova la playlist per id; se il brano è già tra le sue `tracks` lo rimuove, altrimenti lo inserisce in testa; salva l'intero array di playlist e aggiorna la sidebar

Esiste anche una costante speciale `PLAYLIST_FAVOURITES = "favourites"` : non è una playlist reale salvata in `STORAGE_KEY_PLAYLISTS` , ma un id "virtuale" usato per linkare a `playlist.html?id=favourites` e mostrare lì i preferiti **come se fossero una playlist** (gestito in `playlist.js` , sezione 9.5).

8.5 Menu "+" aggiungi a playlist (righe 891-981)

Questo è un componente UI riutilizzato identicamente in `home.js`, `search.js`, `album.js`, `artist.js` e `playlist.js` tramite la funzione condivisa `makeAddButton(track, className)`, che crea un bottone "+" con un listener di click. Al click:

1. `e.stopPropagation()` impedisce che il click "passi" alla card/riga sottostante (che altrimenti avvierebbe la riproduzione)
2. `closePIMenu()`, variabile globale `openPIMenu` — garantisce che ne sia aperto **al massimo uno alla volta** in tutta la pagina)
3. `buildPIMenu(track)`: un riquadro (`.pl-menu`) con intestazione `Aggiungi "titolo" a:`, un bottone per ciascuna playlist esistente (click → `toggleTrackInPlaylist` + chiusura), e in fondo un bottone "+ Crea nuova playlist" che si **trasforma in un campo di testo** al click (`btnCreate.replaceWith(input)`): premendo Enter con testo non vuoto crea la playlist (`createPlaylist`) e ci aggiunge subito il brano; Escape chiude senza creare nulla
4. `document.body` (non al bottone, per evitare problemi di overflow/clipping) e posizionato manualmente in coordinate assolute calcolate da `btn.getBoundingClientRect()`
5. `{ once: true }` sia a uno scroll (anche qui `{ once: true }`): la posizione del menu è calcolata una sola volta all'apertura, quindi se l'utente scrolla la pagina il menu "si scollegherebbe" visivamente dal bottone — la soluzione adottata è semplicemente chiuderlo non appena si rileva uno scroll, invece di ricalcolarne la posizione dinamicamente

9. Rendering della sidebar (righe 983-1124)

9.1 `renderSidebarFavourites()` (righe 991-1030)

Seleziona **entrambi** i contenitori della lista preferiti con un solo `querySelectorAll("#sidebar-favs-list, #mobile-favs-list")` — cioè sia la sidebar desktop/tablet sia il pannello offcanvas mobile vengono popolati identicamente in un solo passaggio, clonando `#tmpl-fav-item` per ogni brano preferito (cover, titolo, artista) e agganciando un click che richiama `window.player.play(track)`. Se non ci sono preferiti, mostra un singolo `<li>` placeholder con il testo "Nessuno ancora".

9.2 `renderSidebarPlaylists()` (righe 1033-1076)

Stessa logica a doppio contenitore (`#sidebar-playlists-list`, `#mobile-playlists-list`), clonando `#tmpl-playlist-item`. La prima voce è sempre fissa: "Brani che ti piacciono", che linka a `playlist.html?id=favourites` (usa `PLAYLIST_FAVOURITES`); seguono le playlist reali dell'utente, ciascuna linkata a `playlist.html?id=<suo-id>`. Se non esiste nessuna playlist reale, dopo la voce fissa dei preferiti viene mostrato comunque il placeholder "Nessuna playlist".

9.3 `initPage()` (righe 1110-1124)

Funzione di bootstrap chiamata come prima riga di ogni script di pagina (`home.js`, `search.js`, ecc.). In ordine:

1. crea `new Player()` e chiama `player.mount()` (costruisce il footer)

2. `window.player = player`
3. `migrateOldPlaylist()` (idempotente, vedi 8.3)
4. `renderSidebarFavourites()` e `renderSidebarPlaylists()` per popolare subito la sidebar con i dati persistiti
5. `myFunction()`
6. `player`, che ogni script di pagina assegna alla propria variabile locale `const player = initPage();`

10. Pagina Home — home.js

La Home mostra fino a 5 "righe" orizzontali scrollabili di card, ciascuna in una `<section class="row-scroller">` già presente nell'HTML con id fisso (`#row-history` , `#row-favourites` , `#row-pop` , `#row-rock` , `#row-hits`), tutte inizialmente nascoste con `d-none` .

10.1 Caricamento dati (loadHome, righe 78-114)

1. Nasconde preventivamente tutte le 5 righe.
2. Legge sincronamente da localStorage `getHistory()` e `getFavourites()` : se non vuote, le rende immediatamente con `renderRow(...)` (nessuna chiamata di rete necessaria, quindi appaiono istantaneamente).
3. Lancia in **parallelo** (`Promise.all`) tre chiamate a `fetchTracksByTerm` : termine "pop" filtrato per genere "Pop", termine "rock" filtrato per genere "Rock", e termine "pop italiano" filtrato per genere "Pop" con parametro aggiuntivo `country=IT` (per ottenere risultati di mercato italiano, mostrati come riga "Suggerimenti hits").
4. Ogni riga con risultati non vuoti viene resa; le righe rimaste vuote restano nascoste.
5. Un blocco `catch` a livello globale, in caso di errore imprevisto in tutto il flusso, sostituisce l'intero contenuto di `.home` con un messaggio d'errore.

10.2 fetchTracksByTerm(term, genre, country) (righe 49-67)

Interroga `search?term=...&media=music&entity=song&limit=50` (più `&country=IT` se passato), poi **filtra lato client** i risultati confrontando `primaryGenreName` (case-insensitive, con `.includes()` , quindi un match parziale) col genere richiesto, e infine taglia il risultato filtrato ai primi 25 elementi, mappandoli in istanze `Track` . In caso di errore di rete ritorna un array vuoto (la riga semplicemente non appare).

10.3 buildCard(track, currentTracklist) (righe 163-198)

Clona `#tmpl-card` e popola: immagine, titolo, e un link `.card-sub` (testo = nome artista, `href` verso `artist.html?id=...` , con `stopPropagation` sul click per non innescare anche la riproduzione del brano). Imposta `card.dataset.id = track.id` — fondamentale perché è la chiave usata da `Player.updateNowPlayingUI()` per ritrovare ed evidenziare la card corretta. Aggancia il cuoricino preferiti (stato iniziale sincronizzato con `isFavourite`), il bottone "+" playlist (`makeAddButton`), e due modi equivalenti per avviare la riproduzione: click sul bottone play dedicato oppure click su un punto qualsiasi della card — entrambi chiamano `window.player.play(track, currentTracklist)` , passando **l'intera riga di provenienza** come tracklist di contesto, in modo che next/prev/shuffle scorrano quella riga specifica.

10.4 renderRow() e initRowNav() (righe 200-261)

`renderRow` distingue tra le 5 righe "conosciute" (popola il container `.d-flex` già esistente dentro la sezione con id fisso) e un eventuale titolo non mappato in `ROW_IDS` (in tal caso costruirebbe una sezione intera da zero — ramo di fatto non utilizzato dai 5 titoli effettivamente passati da `loadHome` , ma presente come fallback generico). `initRowNav()` aggancia ai bottoni freccia `.row-btn-prev` / `.row-btn-`

`next` di ogni `.row-scroller` uno scroll orizzontale fluido (`scrollBy({behavior:"smooth"})`) calcolato dinamicamente sulla larghezza della prima card più il gap Bootstrap di 16px — viene chiamata **prima** di `loadHome()` così i listener sono già pronti quando le card vengono effettivamente inserite nel DOM.

10.5 Ricerca dalla Home

L'input `#search-input` nella topbar, se digitato (debounce 400ms), salva il termine in `STORAGE_KEY_LAST_SEARCH` e naviga subito a `search.html` — la Home non ha una propria UI di risultati di ricerca, delega sempre alla pagina dedicata.

11. Pagina Search — `search.js`

Gestisce tre liste di risultati indipendenti, ciascuna nella propria `<section>` con attributo `hidden` di default: `#row-tracks / #grid-tracks`, `#row-albums / #grid-albums`, `#row-artists / #grid-artists`.

`doSearch(term)` : se il termine è vuoto, nasconde e svuota tutte e tre le sezioni tramite `showRow(section, grid, [], renderFn)`; altrimenti salva subito il termine in `STORAGE_KEY_LAST_SEARCH` e lancia in **parallelo** (`Promise.all` + `fetchJSON`, non `fetchTracksByTerm` come in `home.js`) tre richieste: `entity=song&limit=20`, `entity=album&limit=8`, `entity=musicArtist&limit=8`. Ogni sezione viene mostrata (`section.hidden = items.length === 0`) solo se ha almeno un risultato.

Tre funzioni di rendering distinte costruiscono card diverse a seconda del tipo: `renderTrackCard` (cliccabile ovunque per `player.play`, con cuoricino preferiti e "+" playlist, cover ingrandita via `bigArt`), `renderAlbumCard` (click sulla card naviga a `album.html?id=...`, con link separato sull'artista che naviga invece ad `artist.html`), `renderArtistCard` (mostra un'emoji 🎤 al posto della cover, click naviga a `artist.html?id=...`).

Al caricamento della pagina, se `STORAGE_KEY_LAST_SEARCH` contiene un valore (impostato da un'altra pagina o da una ricerca precedente), l'input viene precompilato e `doSearch` viene eseguita immediatamente — questo è il motivo per cui digitare nella searchbar di Home/Album/Artist e arrivare su `search.html` mostra subito i risultati pertinenti senza dover ridigitare nulla.

12. Pagina Album — `album.js`

`loadAlbum()` legge `id` dalla query string; se assente, o se la lookup API non restituisce risultati, o se restituisce la sola collection senza tracce, mostra "Album non trovato" tramite `showNotFound()` e interrompe il flusso. Altrimenti interpreta `data.results[0]` come l' Album e `data.results.slice(1)` come le sue `Track` (questo è il contratto della lookup API di iTunes: il primo risultato è sempre la collection, i successivi le tracce).

`renderHero` costruisce: cover grande (`bigArt`), etichetta "ALBUM", titolo, sottoriga con link all'artista + anno (estratto da `releaseDate` con `new Date().getFullYear()`) + numero brani + durata totale (somma di tutti i `durationMs`, formattata con `formatTime`), un bottone play grande che avvia la **prima traccia passando l'intero array tracce come tracklist** (così next/prev scorrono l'intero album), e un cuore "salva album" la cui logica è particolare: è considerato "acceso" solo se **tutte** le tracce dell'album sono già nei preferiti (`isAlbumFavourite`); al click, se l'album è già tutto preferito lo rimuove

integralmente, altrimenti aggiunge tutte le tracce non ancora presenti — quindi click singolo che opera sull'intero album in blocco, non sulla singola traccia.

`renderTracklist` costruisce una riga per traccia (numero, titolo, durata, cuore preferiti, "+" playlist), con `data-id` sulla riga per l'evidenziazione "in riproduzione", e il click sulla riga avvia `player.play(track, tracks)` passando sempre l'intero album come contesto di navigazione.

13. Pagina Artist — `artist.js`

Struttura quasi identica ad `album.js`: `loadArtist()` legge `id`, interroga `lookup?id=ID&entity=song&limit=15`, tratta `results[0]` come `Artist` e il resto come le sue top track. La differenza principale è nella hero: non esiste un anno/numero brani reali, quindi viene mostrato un numero di "ascoltatori mensili" **generato casualmente** (`Math.floor(Math.random() * 5_000_000)`), formattato con separatori delle migliaia tramite `toLocaleString("it-IT")` — un dato puramente cosmetico, non fornito dall'API e non persistente, quindi **cambia ad ogni reload della pagina**. Il resto (botone play sulla prima track passando l'intero set come tracklist, tracklist con cuore/+click) è strutturalmente identico ad `album.js`.

14. Pagina Playlist — `playlist.js`

Questa è l'unica pagina tra le cinque che non dipende da una fetch verso l'API iTunes per il proprio contenuto principale: legge tutto da `localStorage`. `loadPlaylist()` legge `id` dalla query string e definisce una funzione interna `render()` che viene **richiamata ogni volta che lo stato sottostante cambia** (rimozione di un preferito, rimozione di un brano dalla playlist) — pattern di "re-render totale" invece di patch puntuali del DOM.

Se `id === PLAYLIST_FAVOURITES` (cioè "favourites"), il titolo mostrato è "Brani che ti piacciono" e i brani sono `getFavourites()`; altrimenti cerca la playlist reale con `getPlaylistById(id)` — se non la trova, "Playlist non trovata" e stop.

`renderHero`: cover presa dal primo brano della lista (se presente, altrimenti un'emoji 🎵 generica), conteggio brani con singolare/plurale corretto ("1 brano" vs "N brani"), durata totale, bottone play (disabilitato se la lista è vuota) che avvia il primo brano con l'intera lista come tracklist. Il bottone cestino "🗑" per eliminare la playlist è mostrato **solo se non è la vista preferiti** (`!isFavourites`): al click chiede conferma con `confirm()` nativo del browser, poi chiama `deletePlaylist(id)` e reindirizza a `index.html`.

`renderTracklist` ha un comportamento del bottone d'azione che **cambia in base al contesto**: nella vista preferiti, ogni riga ha il "+" standard per aggiungere il brano a una playlist (`makeAddButton`); in una playlist reale, ha invece una "X" dedicata che rimuove quel brano da **quella specifica playlist** (`toggleTrackInPlaylist`) e richiama `render()` per far scomparire subito la riga. Anche il cuore preferiti, qui, richiama sempre `render()` dopo il toggle: questo significa che se si rimuove un brano dai preferiti mentre si sta visualizzando la playlist "Brani che ti piacciono", la riga scompare immediatamente dalla vista, perché l'intera funzione `render()` ri-legge `getFavourites()` da capo.

La searchbar in questa pagina usa un meccanismo diverso dalle altre quattro: niente debounce, ma un listener su `keypress` che intercetta solo il tasto `Enter` per salvare la query e navigare a `search.html`.

15. Topbar e offcanvas mobile

La `<header class="topbar">`, identica su tutte le pagine, è `position: sticky; top: 0` con `backdrop-filter: blur(20px)` e sfondo semitrasparente — resta visibile durante lo scroll del contenuto principale. Contiene: bottone home (link a `index.html`), input di ricerca, e — alternativamente, secondo le classi utility Bootstrap — il pulsante utente "  Utente" (`d-none d-sm-block`, quindi visibile solo da 576px) oppure il bottone hamburger (`d-sm-none`, quindi visibile solo sotto 576px) che apre, tramite gli attributi `data-bs-toggle="offcanvas"` / `data-bs-target="#mobile-menu"` gestiti nativamente da Bootstrap JS, il pannello laterale `#mobile-menu`.

Il pannello offcanvas **specchia** manualmente il contenuto della sidebar desktop: stesso brand, stesso link Home, stesse sezioni collassabili Preferiti/Playlist (con gli stessi id `#mobile-favs-list` / `#mobile-playlists-list` popolati dalle stesse funzioni `renderSidebarFavourites` / `renderSidebarPlaylists` che già gestiscono in un colpo solo sia la sidebar desktop che questo pannello, come visto in sezione 9). Le sezioni collassabili usano i componenti Bootstrap `collapse` nativi (`data-bs-toggle="collapse"`), senza JavaScript custom.

16. L'elemento `<audio>` e il ciclo di vita della riproduzione

Ogni pagina HTML include un singolo `<audio id="audio-element"></audio>` vuoto, senza attributo `src` iniziale né `controls` (l'interfaccia visibile è interamente quella custom costruita da `Player.mount()`, l'elemento nativo resta invisibile e serve solo come motore di decodifica/riproduzione). Il flusso completo, dal click dell'utente al suono:

1. L'utente clicca una card, una riga di tracklist, o un bottone play grande → viene chiamato `window.player.play(track, tracklist)`
2. `audio.src` viene impostato sull'URL di anteprima di 30 secondi fornito da Apple (`track.previewUrl`, tipicamente un file `.m4a`)
3. `audio.play()` avvia il caricamento e la riproduzione (asincrono: il browser scarica abbastanza buffer per iniziare)
4. `timeupdate` (scatenato molte volte al secondo dal browser durante la riproduzione) aggiorna progress bar e tempo corrente
5. al termine naturale del brano, l'evento `ended` decide se passare al successivo (se c'è una tracklist con più elementi) o fermarsi
6. se `isRepeat` è attivo, `audio.loop = true` impedisce che `ended` scatti affatto per il singolo brano: il browser lo ricomincia da capo autonomamente

Il volume effettivo percepito è il prodotto di due fattori indipendenti e non sovrapposti: `audio.volume` (0-1, controllato dalla barra software, attiva solo da 576px in su) e il volume di sistema/hardware del dispositivo (sempre attivo, unico canale di controllo disponibile sotto i 576px una volta che `audio.volume` è forzato a 1).

17. Esempio di flusso dati end-to-end

Per fissare come tutti i pezzi collaborano, ecco la sequenza completa quando un utente, sulla Home, clicca play su una card nella riga "Suggerimenti pop":

1. **Al caricamento della pagina:** `home.js` chiama `initPage()` (`common.js`) → crea `Player`, monta il footer, espone `window.player`, migra eventuali vecchie playlist, popola sidebar (preferiti/playlist) leggendo `localStorage`, attiva i filtri badge.
2. `home.js` chiama `loadHome()` → in parallelo scarica da iTunes i brani per "pop" (filtrati per genere), li trasforma in istanze `Track` tramite il costruttore che rimappa i campi dell'API, e chiama `renderRow("Suggerimenti pop", popTracks)`.
3. `renderRow` trova la sezione `#row-pop` già presente nell'HTML, la rende visibile (rimuove `d-none`) e popola il suo container `.d-flex` con `tracks.map(track => buildCard(track, tracks))` — ogni card riceve **l'intera riga** come futura tracklist di contesto.
4. `buildCard` clona `#tmpl-card`, imposta `card.dataset.id`, popola immagine/titolo/link artista, e collega due percorsi equivalenti di click (bottone play dedicato o intera card) entrambi a `window.player.play(track, currentTracklist)`.
5. **Al click dell'utente:** `Player.play()` imposta `audio.src` e avvia `audio.play()`, aggiorna l'intero footer (cover/titolo/artista/durata/icona), salva il brano in cronologia (`addToHistory` → scrive `STORAGE_KEY_HISTORY`), e chiama `updateNowPlayingUI()` che evidenzia in verde la card appena cliccata cercandola per `data-id` tra tutte le card della pagina.
6. Durante la riproduzione, `timeupdate` aggiorna in tempo reale la barra di progresso nel footer; se l'utente trascina la barra del volume, `setVolume()` applica il cambiamento e lo persiste subito in `STORAGE_KEY_VOLUME` (solo se il viewport è $\geq 576\text{px}$, altrimenti la barra è invisibile e il volume resta fissato a 1, governato dai tasti fisici).
7. Se l'utente clicca "successivo" o il brano termina naturalmente, `next()` consulta `currentTracklist` (l'intera riga "Suggerimenti pop" salvata al click) per decidere il prossimo brano, rispettando lo stato shuffle/repeat persistito.
8. Se l'utente clicca il cuoricino sulla card, `toggleFavourite` aggiorna `STORAGE_KEY_FAVOURITES` e richiama immediatamente `renderSidebarFavourites()`, che ripopola sia la sidebar desktop sia il pannello offcanvas mobile in un solo passaggio.

18. Riepilogo delle chiavi localStorage

Chiave	Tipo dato	Scritta da	Letta da
<code>epitunes_history</code>	Array<Track> JSON, max 12	<code>addToHistory</code> (chiamata da <code>Player.play</code>)	<code>getHistory</code> (<code>home.js</code>)
<code>epitunes_favourites</code>	Array<Track> JSON	<code>toggleFavourite</code>	<code>getFavourites</code> , <code>isFavourite</code> (tutte le pagine)
<code>epitunes_playlist</code>	Array<Track> JSON (legacy)	<code>togglePlaylist</code> (helper legacy, non più collegato a	<code>migrateOldPlaylist</code> (una sola volta, poi

		UI attiva)	rimossa)
<code>epitunes_playlists</code>	Array<{id,name,tracks}> JSON	<code>createPlaylist</code> , <code>deletePlaylist</code> , <code>toggleTrackInPlaylist</code> , <code>migrateOldPlaylist</code>	<code>getPlaylists</code> , <code>getPlaylistById</code> , sidebar, playlist.js
<code>epitunes_last_search</code>	Stringa	home.js, album.js, artist.js, playlist.js (su digitazione/Enter nella searchbar), search.js (su ogni ricerca)	search.js (al caricamento, per ripristinare l'ultima query)
<code>epitunes_volume</code>	Stringa numerica "0".. "1"	<code>Player.setVolume</code> (solo da 576px in su)	<code>Player.mount</code> → <code>applyVolumeForViewport</code>
<code>epitunes_shuffle</code>	"true"/"false"	<code>Player.toggleShuffle</code>	<code>Player</code> (costruttore)
<code>epitunes_repeat</code>	"true"/"false"	<code>Player.toggleRepeat</code>	<code>Player</code> (costruttore, imposta <code>audio.loop</code>)

19. Dipendenze esterne caricate da CDN

Libreria	Versione	Provenienza	Uso effettivo nel progetto
Bootstrap CSS	5.3.3	jsdelivr CDN	Griglia (<code>row</code> , <code>col-*</code>), utility (<code>d-none</code> , <code>d-flex</code> , <code>d-sm-block</code> , ecc.), componenti <code>offcanvas</code> e <code>collapse</code>
Bootstrap JS bundle	5.3.3	jsdelivr CDN	Comportamento JS di <code>offcanvas</code> e <code>collapse</code> (nessun JS custom scritto per questi componenti)
Ionicons	7.1.0	unpkg CDN (modulo ES + fallback <code>nomodule</code>)	Icone <code><ion-icon></code> in sidebar, topbar, card della Home (play/pause, home, menu, cuore, note musicali, freccette carosello)
iTunes Search/Lookup API	—	itunes.apple.com	Unica fonte di dati per brani/album/artisti/anteprime audio in tutto il progetto

20. Sintesi architetturale

EpiTunes è organizzato secondo un pattern "**shell condivisa + pagina specializzata**": ogni file HTML ripete (con piccolissime differenze) lo stesso scheletro di layout (sidebar, topbar, footer player, offcanvas), e `common.js` fornisce un'unica implementazione condivisa di tutto ciò che è trasversale — modello dati, player audio, persistenza locale, sidebar dinamica, sistema playlist — mentre ciascun file `*.js` specifico per pagina si occupa solo della porzione di contenuto centrale che cambia (la griglia di righe in Home, i tre gruppi di risultati in Search, la hero+tracklist in Album/Artista/Playlist). Non essendoci alcun bundler né alcun sistema di moduli, l'intera "API interna" del progetto (funzioni, classi,

costanti di `common.js`) vive come semplice scope globale del browser, condiviso per ordine di caricamento degli script nell'HTML.